

# Conceptual Design and Architectural Blueprint for a Constraint Structure Query Language

Driven by Dean Kulik

June 2026

## The Paradigm Shift: From Extensional Data to Intensional Constraints

The historical trajectory of data interrogation has been overwhelmingly dominated by extensional models. In relational database management systems, query languages such as SQL operate over finite, explicitly materialized sets of tuples. A database query traverses physical or indexed storage to retrieve, filter, and aggregate concrete objects. However, as computational demands have expanded into the realms of formal verification, program synthesis, symbolic execution, and abstract interpretation, a fundamental gap has emerged. In these advanced domains, data is not stored as discrete records; rather, it is represented intensionally as infinite or highly complex state spaces governed by mathematical constraints.

Satisfiability Modulo Theories (SMT) solvers, most notably Microsoft Research's Z3, have become the primary engines for interrogating these intensional spaces. SMT solvers decide the satisfiability of logical formulas with respect to combinations of background theories, such as linear arithmetic, bit-vectors, arrays, and uninterpreted functions. Yet, interacting with these powerful solvers currently requires imperative API calls or low-level constraint scripting languages like SMT-LIB2. These interfaces lack the composability, abstraction, and declarative elegance of a native query language. Furthermore, formulating questions about the structural properties of a constraint system—such as asking which variables are rigidly fixed, which constraints cause contradictions, or what subsets of the logic actually influence a target variable—requires ad-hoc, programmatic wrapping around the solver's core algorithms.

The architectural blueprint presented in this report establishes a formal conceptual design for a novel query language designed exclusively to interrogate constraint structures rather than computing objects. This language enables the declarative extraction of boundaries, backbones, and reachability spaces within highly dimensional constraint topologies. By treating the network of constraints as a queryable manifold, this

framework translates the complex mechanics of SMT solving, abstract interpretation, and program slicing into a cohesive generalized syntax centered on a novel SELECT/FROM/READING paradigm.

To validate the semantic fidelity of this grammar, the design mandates a rigorous calibration methodology. Deterministic, cryptographically dense structures, such as the Secure Hash Algorithm (SHA) family, are utilized as the ground-truth testing domain. Once the language proves capable of isolating precise cones of influence and invariants within the strict confines of cryptographic bit-vector arithmetic, the architecture can be generalized to interrogate the "messier" constraint domains of neural network verification, role-based access control (RBAC), and continuous hybrid systems.

**Epistemology of Constraint Interrogation: Analogies to Spigot Algorithms**

To understand the mechanics of querying an infinite constraint space without enumerating all possible states, it is highly instructive to examine the epistemological shift that occurred in mathematics with the discovery of digit-extraction algorithms. Prior to 1995, calculating a specific digit of a transcendental number like  $\pi$  required computing all preceding digits. This linear dependence meant that querying the  $n$ -th digit carried an enormous computational and memory burden.

This paradigm was shattered by the discovery of the Bailey–Borwein–Plouffe (BBP) formula, which was identified using the PSLQ integer relation algorithm. The PSLQ algorithm is designed to find non-trivial integer relations among high-precision floating-point numbers. Using this algorithm, researchers uncovered a formula that functions as a spigot algorithm, enabling the direct computation of individual hexadecimal or binary digits of  $\pi$  without requiring the calculation of any preceding digits. The BBP formula relies on a structure featuring a geometric factor of  $1/16^k$ , combined with a linear combination of rational terms, allowing the  $n$ -th digit to be computed efficiently using modular exponentiation. By reducing terms modulo  $8k + 1$  and similar linear denominators, the algorithm isolates the fractional part associated with the specific digit, completely discarding the irrelevant integer parts generated by previous positions.

An efficient constraint query language must operate on the exact same philosophical principle. When a user queries a constraint system with millions of variables, forcing an SMT solver to compute or enumerate the entire state space to find a specific invariant is computationally intractable. The query language must leverage structural reductions to target only the relevant "slice" of the problem, extracting the necessary truth value of a single variable (analogous to a single hexadecimal digit) without resolving the entire formula.

| Algorithmic Domain    | Traditional Approach                                               | Extraction-Based Paradigm       | Mechanism of Reduction                                                     |
|-----------------------|--------------------------------------------------------------------|---------------------------------|----------------------------------------------------------------------------|
| Transcendental Digits | Compute all $n - 1$ digits sequentially to find the $n$ -th digit. | BBP Formula / Spigot Algorithms | Modular arithmetic isolates fractional parts; independent term evaluation. |

| Algorithmic Domain    | Traditional Approach                                            | Extraction-Based Paradigm       | Mechanism of Reduction                                                    |
|-----------------------|-----------------------------------------------------------------|---------------------------------|---------------------------------------------------------------------------|
| Constraint Structures | Enumerate all valid models to find invariant system properties. | Constraint Query Language (CQL) | Dependency slicing, Cone of Influence isolation, and Backbone extraction. |

This digit-extraction methodology forms the philosophical bedrock of the proposed query language. The architecture is designed to map declarative user queries into targeted reductions, ensuring that the backend solver only processes the mathematical bounds strictly necessary to return the requested coordinate or invariant.

### Architectural Formalization of the Five Core Primitives

The expressiveness and computational efficiency of the constraint query language rely entirely on the formalization of five foundational primitives: Direction, Boundary, Wedge (Reach), Residue, and Resolution. These primitives abstract the highly complex operations of formal methods into composable semantic building blocks.

#### Direction

In traditional databases, querying is an atemporal retrieval of existing facts. In constraint structures, relationships represent causal, temporal, or mathematical dependencies. The Direction primitive specifies the flow of logical deduction, dictating whether the query traverses the constraint graph from causes to effects or from effects back to causes. This concept is formally derived from the theories of forward and backward program slicing, as well as reachability semantics.

A forward direction query interrogates the reachable state space or the logical consequences of a given state. In SMT terminology, given a precondition or initial state  $I$ , it asks what must necessarily follow. This aligns with forward abstract executions, which mimic testing by deriving postconditions. Conversely, a backward direction query begins with a target state, postcondition, or failure criterion  $C$ , and identifies the conditions that lead to  $C$ . Backward analysis is utilized heavily in software verification to derive necessary preconditions or to identify the root causes of vulnerabilities and crashes.

Formally, if a constraint system is represented as a state transition relation  $T(s, s')$ , a forward query computes the image  $Post(I) = \{s' \mid \exists s \in I. T(s, s')\}$ , while a backward query computes the pre-image  $Pre(C) = \{s \mid \exists s' \in C. T(s, s')\}$ . In the context of the query language, setting the Direction instructs the internal slicing algorithms on how to traverse the control and data dependency edges. For example, a backward slice consists of all statements and control predicates that directly or indirectly affect the values computed at the query target. By exposing Direction as a primitive, the language allows analysts to seamlessly pivot between predictive modeling (forward) and root-cause analysis (backward).

#### Boundary

The Boundary primitive dictates the mathematical limits and the nature of the constraint space being interrogated. It defines both the underlying logic fragment (the SMT theory) and the level of precision required. The theoretical basis for the Boundary primitive is found in Abstract Interpretation and the formulation of Galois connections.

Constraint solving is highly sensitive to the underlying mathematical theory. For instance, Linear Real Arithmetic (LRA) is efficiently decidable using the Dual Simplex algorithm, whereas Non-Linear Real Arithmetic (NRA) requires highly expensive Model-based Cylindrical Algebraic Decomposition (CAD) and is generally undecidable over integers. When interrogating highly complex or non-linear spaces, finding an exact solution may be impossible or computationally intractable. The Boundary primitive allows the query to specify an abstract domain  $A$  (e.g., intervals, convex polyhedra, octagons, or congruence relations) that safely over-approximates the concrete domain  $C$ .

This relationship is mathematically formalized via a Galois connection  $\langle \alpha, \gamma \rangle$ , where  $\alpha: C \rightarrow A$  is the abstraction function and  $\gamma: A \rightarrow C$  is the concretization function. A valid Boundary ensures that for any concrete element  $c \in C$  and abstract element  $a \in A$ :

$$\alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_C \gamma(a)$$

By defining the Boundary, the query explicitly tells the backend solver whether to attempt exact resolution (e.g., bit-blasting a cryptographic function) or to utilize an over-approximation (e.g., polyhedral abstraction) to guarantee termination while preserving soundness.

| Logic Fragment / Domain          | Abstraction Level             | Typical Solver Algorithm       | Example Constraint                            |
|----------------------------------|-------------------------------|--------------------------------|-----------------------------------------------|
| Linear Integer Arithmetic (LIA)  | Concrete (Exact)              | CutSat, Branch and Bound       | $a + 3b \leq 3$<br>[cite: 20]                 |
| Polynomial Real Arithmetic (NRA) | Concrete (Exact)              | Incremental Linearization, CAD | $x^2 + y^2 < 1$<br>[cite: 20]                 |
| Interval Domain                  | Abstract (Over-approximation) | Abstract Interpretation        | $x \in [0,10]$<br>[cite: 18]                  |
| Convex Polyhedra                 | Abstract (Relational)         | Abstract Interpretation        | $x + y \leq 10 \wedge x \geq 0$<br>[cite: 18] |

## Wedge (Reach)

The Wedge primitive represents the active slice of the constraint network that is relevant to the query. In large-scale industrial systems, submitting an entire formula  $\Phi$  to a solver is grossly inefficient. The Wedge isolates the Cone of Influence (COI), which is the subset of variables and constraints that can directly or indirectly affect the query target.

Derived from the extensive literature on program slicing, a slicing criterion is typically defined as a tuple  $C = \langle \ell, X \rangle$ , where  $\ell$  is a specific point or constraint in the program and  $X$  is a set of variables whose values are of interest at  $\ell$ . The Wedge primitive extracts the Program Dependence Graph (PDG) or System Dependence Graph (SDG), which contains nodes corresponding to statements and control predicates, and edges corresponding to data and control dependencies.

For a backward Wedge  $W_B(C)$  over a dependency graph  $G = (V, E)$ , the slice includes all nodes that can reach the criterion:

$$W_B(C) = \{v \in V \mid v \xrightarrow{*} \ell \text{ and modifies } X\}$$

Isolating the Wedge ensures that the query language dynamically reduces the search space before invoking the backend SMT solver. This mechanism is mathematically critical. Most SAT and SMT techniques are computationally very sensitive to the COI size. By identifying the Wedge, the query language strips away irrelevant constraints, performing a structural reduction analogous to predicate pushdown in relational databases, but applied to multi-dimensional variable dependencies.

## Residue

Not all queries are directed at satisfiable, well-formed constraint spaces. In heavily regulated or overconstrained systems, the goal is frequently to discover exactly *why* the constraints contradict one another. The Residue primitive is engineered to extract the irreducible core of a logical contradiction, transforming a binary "unsatisfiable" result into actionable insight.

When a formula  $\Phi$  is unsatisfiable (UNSAT), a standard solver simply returns an error or a failure state. The Residue primitive commands the solver to extract a Minimally Unsatisfiable Subformula (MUS), commonly known as an Unsatisfiable Core. An MUS  $\Phi_{core} \subseteq \Phi$  is defined such that  $\Phi_{core}$  is unsatisfiable, but for any individual constraint  $\phi \in \Phi_{core}$ , the subset  $\Phi_{core} \setminus \{\phi\}$  is satisfiable. The core provides the exact combination of conflicting rules.

Conversely, the Residue primitive can also target Maximal Satisfiable Subsets (MSS) or Minimal Correction Sets (MCS). An MCS represents the smallest number of constraints that must be removed, negated, or relaxed to render the remaining formula satisfiable. This extraction is achieved through resolution proofs generated by solvers using the Conflict-Driven Clause Learning (CDCL) architecture. The Residue primitive is absolutely essential for debugging conflicting corporate access policies, locating post-silicon faults in integrated circuits, and identifying the structural root of an anomaly where the system has been over-constrained.

## Resolution

The Resolution primitive extracts the deterministic truth from a valid constraint space. While a standard SMT query simply returns a single, arbitrary satisfying model (a point in the space), the Resolution primitive can be elevated to extract the *Backbone* of the formula, revealing the invariant properties of the system.

The backbone of a satisfiable formula  $\Phi$  is the set of literals that evaluate to True in every satisfying assignment of  $\Phi$ . Formally, a literal  $l$  is a backbone literal if and only if:

$$\forall M \models \Phi, M(l) = \text{True}$$

Identifying backbone variables (also known as frozen variables or necessary assignments) is a historically demanding computational task. It is co-NP complete to decide if a literal is in the backbone of a formula. The size of the backbone is strongly associated with the hardness of the corresponding propositional problem, with phase transitions occurring in random 3-SAT problems where the backbone size rapidly shifts.

The Resolution primitive automates this complex extraction. Instead of returning a fragile, single-point model, a backbone Resolution utilizes advanced algorithmic approaches—such as Single Unit Resolution Backbone (SURB) algorithms, implicant enumeration, or iterative incremental SAT testing with negated assumptions—to return the non-negotiable parameters of the system. By deploying the Resolution primitive, the query language allows a user to ask, "Regardless of how the other variables fluctuate, which configurations are absolutely mandatory for this system to function?"

### Generalized Syntax: The SELECT / FROM / READING Template

To operationalize the theoretical depth of the five primitives, the language relies on a generalized syntax that provides a declarative, intuitive interface to the underlying solver architecture. The template deliberately mimics the cognitive familiarity of SQL but replaces relational algebra with constraint satisfaction mechanisms. The syntax follows a SELECT / FROM / READING structure.

#### Syntax Grammar Definition

The syntax translates higher-order mathematical operations into distinct, composable clauses:

- **SELECT:** Defines the output extraction mechanism, mapping directly to the **Resolution** and **Residue** primitives.
  - *Directives:* MODEL (returns an arbitrary valid assignment), BACKBONE (returns invariant literals across all models), CORE (extracts the MUS for contradictions), MCS (extracts the Minimal Correction Set).
- **FROM:** Defines the **Boundary** and the background theory, specifying the abstraction level and the data types involved in the constraint space.
  - *Directives:* THEORY LRA (Linear Real Arithmetic), THEORY BV[256] (Bit-Vectors of 256 bits), DOMAIN POLYHEDRA (Abstract interpretation bounds), THEORY Strings (String processing).
- **READING:** Defines the **Wedge** (the specific constraints to be evaluated) and the **Direction** of the query, alongside any dynamic assumptions used to prune the search space.

- *Syntax Structure*: READING <Constraint\_Block> [DIRECTION FORWARD | BACKWARD] [ASSUMING <Literal\_List>]

### Structural Translation Mapping

The relationship between the declarative query syntax and the internal SMT and abstract interpretation operations is strictly defined. The parser translates the syntax into specific solver API calls.

| Query Clause    | Core Primitive Assessed | Internal Solver Mechanism                                   | Output Semantics                                                                             |
|-----------------|-------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| SELECT MODEL    | Resolution              | check-sat followed by get-model<br>[cite: 2, 20]            | A specific, non-unique valid coordinate in the state space.                                  |
| SELECT BACKBONE | Resolution              | Implicant enumeration, SURB, or iterative assumptions       | The set of invariant, frozen literals across all valid models.                               |
| SELECT CORE     | Residue                 | UNSAT core extraction via CDCL resolution proofs            | The irreducible constraints causing a mathematical contradiction.                            |
| FROM THEORY     | Boundary                | CDCL(T) theory instantiation (e.g., LIA, NRA, Strings)      | Strict adherence to the mathematical axioms of the chosen fragment.                          |
| FROM DOMAIN     | Boundary                | Galois Connection abstraction ( $\alpha$ )                  | Safe over-approximations mapping intractable concrete variables to intervals/polyhedra.      |
| READING ...     | Wedge                   | Dependency graph parsing, Cone of Influence (COI) reduction | The isolated, active subset of constraints submitted to the solver, ignoring disjoint paths. |

| Query Clause | Core Primitive Assessed | Internal Solver Mechanism                                                 | Output Semantics                                                                                |
|--------------|-------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| DIRECTION    | Direction               | Forward/Backward Program Slicing algorithms                               | Determines the causal chain traversal (preconditions vs. postconditions).                       |
| ASSUMING     | Context                 | Incremental solving via literal assumptions (push/pop or API assumptions) | Temporary state conditions tested without rebuilding the formula tree or losing learned lemmas. |

### Concrete Query Formulation Examples

To demonstrate the applicability of the syntax, consider two distinct verification scenarios.

**Scenario 1: Backbone Extraction in Hardware Verification** An engineer needs to identify which logic gates are invariant (frozen) when a specific output is forced to a high state in a bit-vector circuit. Instead of writing complex C++ Z3 API loops, the query is formulated as:

SQL

SELECT BACKBONE(internal\_wires)

FROM THEORY BV[32]

READING circuit\_logic\_module()

DIRECTION BACKWARD

ASSUMING output\_register = 0xFFFFFFFF

*Mechanistic Analysis:* The query language parser automatically identifies the Wedge by performing a backward slice from output\_register. It loads the reduced circuit into the solver using the Theory of Bit-Vectors. Because SELECT BACKBONE is specified, the engine utilizes incremental iterative SAT solving, assuming the negation of candidate literals to systematically filter out flippable variables until only the true backbone remains. The result reveals the necessary internal states of the circuit.

**Scenario 2: Residue Isolation in Conflicting Scheduling Policies** A system administrator must find out why a set of temporal scheduling constraints is impossible to satisfy.

SQL

SELECT CORE

FROM THEORY IDL

READING

task\_A\_start > task\_B\_end + 5 AND

task\_B\_start > task\_C\_end AND

task\_C\_start > task\_A\_end

*Mechanistic Analysis:* The query invokes the Integer Difference Logic (IDL) fragment, instructing the solver to utilize internal algorithms like Bellman-Ford or Floyd-Warshall. Because the constraints form a cyclical impossibility, the check-sat call yields UNSAT. The language then automatically extracts the resolution proof, isolating the exact subset of rules creating the loop and returning them as the Residue (the core contradiction).

### Ground-Truth Calibration: The Deterministic SHA Methodology

The development of a language capable of managing abstract domains, non-linear arithmetic, and consequence finding requires a meticulously stable ground truth for calibration. Before deploying the grammar against noisy, under-specified, or probabilistically bounded real-world data, the lexical parser, the Wedge isolation algorithms, and the backend solver integration must be thoroughly debugged.

For this purpose, deterministic 1:1 systems—specifically cryptographic algorithms like the Secure Hash Algorithm (SHA-256)—are utilized as the strict, unforgiving calibration domain.

### The Necessity of Cryptographic Functions for Calibration

A cryptographic hash function provides a unique architectural stress test for a constraint solver and query language. It is entirely deterministic and relies exclusively on the Theory of Bit-Vectors (BV). The operations are highly non-linear at the bit level, involving massive chains of modular addition, bitwise XORs, shifts, and rotations. This results in an incredibly dense dependency graph. In a single SHA-256 round, the Cone of Influence (Wedge) rapidly expands to encompass almost all variables due to the cryptographic avalanche effect.

If a query is constructed to find the pre-image of a hash:

SQL

SELECT MODEL(input\_block)

FROM THEORY BV[256]

READING SHA256(input\_block) = 0x5a1f...

DIRECTION BACKWARD

The solver's behavior provides absolute ground truth. The state is either perfectly satisfiable (yielding the exact preimage) or completely unsatisfiable (UNSAT). There are no floating-point rounding errors, no over-

approximations, and no ambiguous logic states. If the query language's slicing algorithm accidentally removes a single necessary constraint while attempting to optimize the Wedge, the cryptographic verification will fail catastrophically.

## **Debugging the Grammar and Incremental Mechanisms**

Using SHA for calibration directly mirrors the utility of the BBP formula discussed earlier. Just as the BBP formula proved that a specific slice of  $\pi$  could be extracted without generating the whole sequence, the query language must prove that its Wedge and Direction primitives can isolate the logical slice of the SHA algorithm required to evaluate a specific bit constraint without unrolling the entire cryptographic execution history into RAM in an unoptimized format.

By relying on Cone of Influence reductions and incremental assumption-based solving, the query language extracts backbone assignments of the cryptographic state. Modern conflict-driven solvers manage learned clauses in databases that expand exponentially. The calibration phase ensures that the language correctly implements SMT push/pop states and assumption literals, allowing the solver to explore local contexts without garbage-collecting lemmas that were derived independently of the assumptions.

If the query language's abstract syntax tree (AST) generation, Wedge reduction, and incremental  $Z_3$  communication can successfully reverse a bounded SHA execution or efficiently generate an unsatisfiable core for an invalid hash configuration without exponential memory explosions, the grammar is formally verified. It guarantees that the mapping from the SELECT/FROM/READING syntax to the CDCL(T) backend is mathematically sound, paving the way for application in domains where the ground truth is not explicitly known.

## **Generalization to Complex and Ambient Constraint Domains**

Once the query language is strictly calibrated against the zero-ambiguity SHA domain, the architecture must be generalized to interrogate "messier" constraint spaces. Real-world systems are rarely 1:1 deterministic mappings. They feature continuous variables, under-specified functions, overlapping heuristic rules, and non-linear real arithmetic. The query language must transition from exact bit-vector satisfiability to domains requiring abstract interpretation and optimization.

### **1. Generalizing to Role-Based Access Control and String Theories**

Modern infrastructure security, such as AWS Zelkova or Teleport RBAC, relies on evaluating access policies composed of identity parameters, resource tags, and string regular expressions. These domains are messy because they involve unbounded strings, wildcards, and overlapping deny/allow constraints.

The query language generalizes to this environment by transitioning the FROM Boundary to String Theories. Advanced SMT solvers, such as  $Z_3$  with its  $Z_3\text{-str}$  extension, treat strings as primitive types, completely bypassing the inherent limitations of encoding strings as fixed-length bit-vectors. The language can interrogate policies using native string operations, including concatenation, sub-string, union, and regex loops.

SQL

SELECT CORE

FROM THEORY Strings

READING

policy\_allow(user, resource) AND

policy\_deny(user, resource)

In this scenario, a wildcard token (e.g., \*) matching any value may cause an overlap where a user is simultaneously granted and denied access. The constraint solver processes the regular expressions natively. If an overlap exists, the Residue (Core) primitive identifies the exact regex contradiction causing the security vulnerability, proving that the organization's policy logic is fractured.

## 2. Generalizing to Neural Network Verification and Abstract Domains

Neural networks represent highly complex, continuous constraint spaces where inputs are multidimensional floating-point arrays and constraints are defined by millions of weighted sums and non-linear activation functions (e.g., ReLU). Querying the safety boundaries of a neural network—such as ensuring an autonomous vehicle model never steers off-road under slight pixel perturbation—is notoriously difficult because traditional SMT solvers struggle immensely with Non-linear Real Arithmetic (NRA).

To query these structures, the language leverages the Boundary primitive to apply Abstract Interpretation. Instead of attempting exact real arithmetic, the query specifies an over-approximation domain. Domains such as DeepPoly, intervals, or octagons are utilized to bound the activation functions within linear relaxations.

SQL

SELECT BACKBONE(safety\_violation)

FROM DOMAIN DeepPoly

READING

neural\_network\_forward\_pass(image) AND

perturbation < 0.05

By utilizing First-Order Logic over the Reals (FO(R)) and weighted structural aggregation (FO(SUM)), the language treats the neural network as a relational constraint graph. If the abstract interpretation proves that the safety violation constraint cannot be satisfied within the polyhedral boundary, the robustness property is mathematically verified. If the result is inconclusive due to the coarseness of the over-approximation, the query framework can trigger Counterexample-Guided Abstraction Refinement (CEGAR), incrementally tightening the Boundary until a definitive model or backbone is achieved.

## 3. Generalizing to Optimization Modulo Theories (MaxSMT)

In many complex domains, such as job-shop scheduling, network topology routing, or resource allocation, constraints are rarely binary. These systems often have multiple valid models, but the user requires the *optimal* configuration. Furthermore, these systems may contain under-specified partial functions, such as division by zero.

The generalized query language accommodates this by transitioning to Optimization Modulo Theories (MaxSMT or  $\nu Z$ ). The SELECT primitive is expanded to support optimization objectives, instructing the solver to find a model that maximizes the number of satisfied soft constraints or minimizes a specific cost function, such as latency or resource utilization. To handle under-specification, the language enforces total functions internally, assigning arbitrary but mathematically consistent values to undefined operations (like  $x/0$ ) to maintain logical soundness during the search space traversal. This ensures that the query language never crashes due to a mathematical singularity in the data, instead returning the optimal valid state bounded by the optimization criteria.

### Architectural Execution Pipeline and Synthesis

To realize this conceptual design, the underlying software architecture must efficiently bridge the declarative user syntax and the highly complex backend solvers. The execution pipeline is systematically structured to ensure that constraint queries are processed with minimal overhead.

First, the lexical parser ingests the SELECT/FROM/READING syntax and generates an Abstract Syntax Tree (AST). Semantic validation ensures that the requested Boundary (e.g., Polyhedra) is mathematically compatible with the constraints in the READING block. Once validated, the static analysis engine computes the data and control dependencies. Utilizing backward and forward slicing, it extracts the Cone of Influence (the Wedge), aggressively discarding any constraints entirely disjoint from the query target. This structural reduction dramatically reduces the variable count before any heavy computation begins.

The reduced Wedge is then translated into the specific solver format (e.g., SMT-LIB2) or passed directly via programmatic APIs (e.g., Z3's Python or C++ bindings). If the query requests Backbones or Cores, the architecture avoids monolithic execution. Instead, it utilizes incremental solving features—such as push(), pop(), and assumption literals—to freeze the state and iteratively test constraints without flushing the solver's learned lemmas and conflict clauses. Finally, the raw solver output (SAT/UNSAT flags, raw models, or unsat cores) is mapped back onto the original query context. The extracted mathematical data is formatted into the specified SELECT structure, providing a clean, human-readable insight into the intensional data space.

The conceptual design outlined in this report proposes a profound paradigm shift in how complex logic and formal verification systems are interrogated. By elevating constraint solving from the realm of imperative scripting to a declarative query language, the cognitive load required to analyze highly dimensional state spaces is drastically reduced. Through the strict formalization of Direction, Boundary, Wedge, Residue, and Resolution, the language provides a robust semantic vocabulary. Anchoring the initial calibration of this grammar in the deterministic domain of cryptographic systems ensures that the core parsing and reduction mechanisms are mathematically flawless. Ultimately, this architecture's reliance on abstract interpretation, Galois connections, and incremental SMT solving allows it to generalize gracefully, providing unparalleled

capabilities to interrogate the structural integrity of neural networks, security policies, and continuous hybrid systems.

### Works cited

1. Z3Prover/z3: The Z3 Theorem Prover - GitHub, <https://github.com/z3prover/z3>
2. Programming Z3 - Stanford CS Theory, <https://theory.stanford.edu/~nikolaj/programmingz3.html>
3. Scaling a Declarative Cluster Manager Architecture with Query Optimization Techniques - VLDB Endowment, <https://www.vldb.org/pvldb/vol16/p2618-rong.pdf>
4. Bellard's formula - Grokipedia, [https://grokipedia.com/page/Bellard's\\_formula](https://grokipedia.com/page/Bellard's_formula)
5. Bailey–Borwein–Plouffe formula - Wikipedia, [https://en.wikipedia.org/wiki/Bailey%E2%80%93Borwein%E2%80%93Plouffe\\_formula](https://en.wikipedia.org/wiki/Bailey%E2%80%93Borwein%E2%80%93Plouffe_formula)
6. Bailey–Borwein–Plouffe formula - Grokipedia, [https://grokipedia.com/page/Bailey%E2%80%93Borwein%E2%80%93Plouffe\\_formula](https://grokipedia.com/page/Bailey%E2%80%93Borwein%E2%80%93Plouffe_formula)
7. Experimental computation as an ontological game changer: The impact of modern mathematical computation tools on the ontology of - CARMA, <https://carmamaths.org/ion/ontology.pdf>
8. Experimental Mathematics and Computational Statistics - OSTI, <https://www.osti.gov/servlets/purl/964411>
9. Happy Early Pi Day! - MATHGEEK.com, <https://mathgeek.com/happy-early-pi-day/>
10. A Survey of Program Slicing Techniques. - SciSpace, <https://scispace.com/pdf/a-survey-of-program-slicing-techniques-1u546ok1jr.pdf>
11. CadiBack: Extracting Backbones with CaDiCaL - Uni Freiburg, <https://cca.informatik.uni-freiburg.de/papers/BiereFroleyksWang-SAT23.pdf>
12. Program Slicing Techniques - Emergent Mind, <https://www.emergentmind.com/topics/program-slicing>
13. Principles of Abstract Interpretation | The MIT Press, <https://mitpress.ubli.sh.com/book/principles-of-abstract-interpretation>
14. Model checking, testing and verification working together | Request PDF - ResearchGate, [https://www.researchgate.net/publication/225247017\\_Model\\_checking\\_testing\\_and\\_verification\\_working\\_together](https://www.researchgate.net/publication/225247017_Model_checking_testing_and_verification_working_together)
15. Slicing Through the Noise: Efficient Crash Deduplication via Trace Reconstruction and Fuzzy Hashing - MDPI, <https://www.mdpi.com/2079-9292/13/23/4817>
16. BinSim: Trace-based Semantic Binary Diffing via System Call Sliced Segment Equivalence Checking, <https://faculty.ist.psu.edu/wu/papers/BinSim.pdf>

17. REPORTRAPPORT,  
[https://faculty.cc.gatech.edu/~harrold/6340/cs6340\\_fall2009/Readings/tip94.pdf](https://faculty.cc.gatech.edu/~harrold/6340/cs6340_fall2009/Readings/tip94.pdf)
18. Abstract interpretation - Wikipedia, [https://en.wikipedia.org/wiki/Abstract\\_interpretation](https://en.wikipedia.org/wiki/Abstract_interpretation)
19. Abstract Interpretation, <https://web.eecs.umich.edu/~weimerw/2014-6610/lectures/weimer-gradpl-12.pdf>
20. Arithmetic | Online Z3 Guide, <https://microsoft.github.io/z3guide/docs/theories/Arithmetic/>
21. Programming Constraint Services with Z3 - Stanford CS Theory,  
<http://theory.stanford.edu/~nikolaj/nus.html>
22. Abstract Interpretation, <https://pages.cs.wisc.edu/~horwitz/CS704-NOTES/10.ABSTRACT-INTERPRETATION.html>
23. Abstract Interpretation, [https://homes.cs.washington.edu/~rjust/courses/CSEP504/ai\\_summary.pdf](https://homes.cs.washington.edu/~rjust/courses/CSEP504/ai_summary.pdf)
24. On Constructivity of Galois Connections, <https://www.math.unipd.it/~ranzato/papers/vmcai18.pdf>
25. US8316332B1 - Constraint minimization method for formal verification - Google Patents,  
<https://patents.google.com/patent/US8316332B1/en>
26. Cone of Influence Analysis at the Electronic System Level Using Machine Learning - AGRA | Uni Bremen, [https://agra.informatik.uni-bremen.de/doc/konf/13\\_dsd\\_systemc\\_coi\\_analysis.pdf](https://agra.informatik.uni-bremen.de/doc/konf/13_dsd_systemc_coi_analysis.pdf)
27. A Vocabulary of Program Slicing-Based Techniques - UPV,  
<https://personales.upv.es/josilga/papers/Vocabulary.pdf>
28. Optimized Model Checking of Multiple Properties, [https://past.date-conference.com/proceedings-archive/2011/DATE11/PDFFILES/IP2\\_02.PDF](https://past.date-conference.com/proceedings-archive/2011/DATE11/PDFFILES/IP2_02.PDF)
29. Verifying Safety Properties of a PowerPCTM1 Microprocessor Using Symbolic Model Checking without BDDs - Carnegie Mellon University, <http://reports-archive.adm.cs.cmu.edu/anon/1999/CMU-CS-99-146.pdf>
30. Looking Inside Literal Blocks: Towards Mining More Promising Learnt Clauses in SAT Solving - ResearchGate,  
[https://www.researchgate.net/publication/312485736\\_Looking\\_Inside\\_Literal\\_Blocks\\_Towards\\_Mining\\_More\\_Promising\\_Learnt\\_Clauses\\_in\\_SAT\\_Solving](https://www.researchgate.net/publication/312485736_Looking_Inside_Literal_Blocks_Towards_Mining_More_Promising_Learnt_Clauses_in_SAT_Solving)
31. Interpolation and SAT-based Model Checking - People @EECS,  
[https://people.eecs.berkeley.edu/~alanmi/courses/2007\\_290N/papers/inter\\_mcmillan\\_cavo3.pdf](https://people.eecs.berkeley.edu/~alanmi/courses/2007_290N/papers/inter_mcmillan_cavo3.pdf)
32. Algorithms for Computing Backbones of Propositional Formulae - Automated Reasoning and Software Reliability - INESC-ID, <https://sat.inesc-id.pt/~mikolas/bb-aicom-preprint.pdf>
33. Predicting Satisfiability at the Phase Transition - UBC Computer Science,  
<https://www.cs.ubc.ca/~kevinlb/papers/2012-PredictingSatisfiability.pdf>

34. Phase transitions and backbones of 3-SAT and maximum 3-SAT - ResearchGate, [https://www.researchgate.net/publication/2412258\\_Phase\\_transitions\\_and\\_backbones\\_of\\_3-SAT\\_and\\_maximum\\_3-SAT](https://www.researchgate.net/publication/2412258_Phase_transitions_and_backbones_of_3-SAT_and_maximum_3-SAT)
35. On Computing Backbones of Propositional Theories - Automated Reasoning and Software Reliability - INESC-ID, <https://sat.inesc-id.pt/~mikolas/ecai10.pdf>
36. BIG Backbones - Nils Froleyks, <https://froleyks.de/assets/pdf/Froleyks%20et%20al.%20-%202023%20-%20BIG%20Backbones.pdf>
37. How incremental solving works in Z3? - Stack Overflow, <https://stackoverflow.com/questions/16422018/how-incremental-solving-works-in-z3>
38. Analyzing Teleport RBAC with Z3: Regexes, queries and formal methods, <https://goteleport.com/blog/z3-rbac/>
39. Z3-str: A Z3-Based String Solver for Web Application Analysis - Computer Science Purdue, <https://www.cs.purdue.edu/homes/xyzhang/Comp/fse13.pdf>
40. Verification of Fuzzy Decision Trees | Request PDF - ResearchGate, [https://www.researchgate.net/publication/368970878\\_Verification\\_of\\_Fuzzy\\_Decision\\_Trees](https://www.researchgate.net/publication/368970878_Verification_of_Fuzzy_Decision_Trees)
41. Query Languages for Neural Networks - Document Server@UHasselt, <https://documentserver.uhasselt.be/bitstream/1942/47507/1/Query%20Languages%20for%20Neural%20Networks.pdf>
42. Query languages for neural networks - arXiv, <https://arxiv.org/pdf/2408.10362>
43. Abstraction Refinement for Large Scale Model Checking - SciSpace, <https://scispace.com/pdf/abstraction-refinement-for-large-scale-model-checking-19t2kpv5tw.pdf>
44. (PDF) Successful SAT Encoding Techniques - ResearchGate, [https://www.researchgate.net/publication/337645339\\_Successful\\_SAT\\_Encoding\\_Techniques](https://www.researchgate.net/publication/337645339_Successful_SAT_Encoding_Techniques)